

KNOWLEDGE BASE

AI · RAG · Embeddings · LLM

Una guida tecnica pratica per capire i componenti fondamentali dei moderni sistemi di intelligenza artificiale generativa

Versione 1.0 · June 2026

Preparato per uso personale — Pasquale Zicchillo

Indice dei Contenuti

1		
.	Fondamenti di Intelligenza Artificiale	3
2		
.	Large Language Models (LLM)	4
3		
.	Embeddings	6
4		
.	Vector Databases	8
5		
.	Retrieval-Augmented Generation (RAG)	9
6		
.	Tecniche RAG Avanzate	11
7		
.	Valutazione e Best Practice	12
G		
.	Glossario Rapido	13

1. Fondamenti di Intelligenza Artificiale

L'**Intelligenza Artificiale** (AI) è un campo dell'informatica che studia sistemi capaci di eseguire compiti che tradizionalmente richiedono intelligenza umana: ragionamento, apprendimento, percezione e generazione di linguaggio naturale.

Gerarchia concettuale

L'AI si struttura in livelli progressivi di specializzazione:

AI	Campo vasto: qualsiasi tecnica che fa esibire ai computer comportamenti intelligenti
Machine Learning	Sottocampo: sistemi che imparano dai dati senza essere esplicitamente programmati
Deep Learning	Sottocampo di ML: reti neurali con molti strati (layer) profondi
NLP	Elaborazione del linguaggio naturale — comprensione e generazione di testo
LLM	Large Language Model — modelli enormi pre-addestrati su testo a larga scala
GenAI	AI Generativa — sistemi che creano contenuti nuovi (testo, immagini, codice)

Paradigmi di apprendimento

- **Supervised Learning**: addestramento su coppie input/output etichettate (es. classificazione email spam).
- **Unsupervised Learning**: il modello trova strutture nei dati senza label (clustering, riduzione dimensionalità).
- **Self-supervised Learning**: il modello genera le proprie label dai dati — es. predire la prossima parola. Fondamentale per i LLM.
- **Reinforcement Learning from Human Feedback (RLHF)**: il modello impara da feedback umani su qualità delle risposte — usato nell'allineamento dei LLM.

Il paradigma Foundation Model

Un **Foundation Model** è un modello pre-addestrato su enormi corpus di dati generici, poi adattato (fine-tuned) a compiti specifici. Questo approccio riduce drasticamente il costo computazionale per ogni nuova applicazione e ha rivoluzionato il panorama AI dal 2020 in poi.

2. Large Language Models (LLM)

I **Large Language Models** sono reti neurali profonde addestrate su miliardi di token di testo con l'obiettivo di modellare la distribuzione statistica del linguaggio umano. Il compito di training di base è la *next-token prediction*: dato un contesto, predire il token più probabile che segue.

Architettura Transformer

Praticamente tutti i LLM moderni si basano sull'architettura **Transformer** (Vaswani et al., 2017). I componenti chiave:

Token Embedding	Mappa ogni token (parola / sottoparola) in un vettore denso ad alta dimensione
Positional Encoding	Aggiunge informazioni sulla posizione, poiché i Transformer non hanno ricorrenza
Multi-Head Attention	Calcola relazioni tra tutti i token del contesto in parallelo — il cuore del Transformer
Feed-Forward Network	Strato densamente connesso applicato indipendentemente su ogni posizione
Layer Normalization	Stabilizza il training normalizzando le attivazioni dopo ogni sotto-layer
Softmax Output	Trasforma i logit finali in probabilità sulla vocabulary per la predizione

Meccanismo di Self-Attention

Dato un input, ogni token calcola tre vettori: **Query (Q)**, **Key (K)** e **Value (V)**. L'attenzione si calcola come:

Formula Self-Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

dove d_k è la dimensione delle chiavi (scaling factor per stabilità del gradiente). Con "multi-head", questo calcolo viene eseguito in parallelo su H sottospazi diversi, poi i risultati vengono concatenati.

Principali famiglie di LLM

GPT-4o / o3	OpenAI — modelli closed-source, API-only, eccellenti per coding e reasoning
Claude 3.5/4.x	Anthropic — focus su safety, context window enorme (200k token), ragionamento
Gemini 2.5	Google DeepMind — multimodale nativo, integrato con Google Search
LLaMA 3.x	Meta — open weights, base per fine-tuning e deployment locale
Mistral / Mixtral	Mistral AI — efficienti, open, MoE architecture (Mixtral 8x7B)
Qwen 2.5	Alibaba — ottimi su lingue asiatiche, open weights, multimodale

Phi-3 / Phi-4	Microsoft — small language models (SLM), alta qualità a bassa dimensione
Gemma 3	Google — open weights, derivati da Gemini, ottimizzati per edge/mobile

Context Window e Token

La **context window** è il numero massimo di token che un LLM può processare in una singola inferenza (input + output). Un token corrisponde approssimativamente a 0.75 parole in inglese (leggermente di più in italiano). Esempi: Claude 3.5 Sonnet → 200k token; GPT-4o → 128k token; LLaMA 3.1 70B → 128k token.

Training Pipeline

- **Pre-training:** next-token prediction su corpus web-scale (Common Crawl, GitHub, libri, Wikipedia...). Costo: milioni di dollari GPU.
- **Supervised Fine-Tuning (SFT):** addestramento su dataset di esempio conversazione (prompt + risposta ideale) per adattare il modello al dialogo.
- **RLHF:** un reward model apprende le preferenze umane; il LLM viene ottimizzato via PPO o DPO per massimizzare la ricompensa.

3. Embeddings

Un **embedding** è una rappresentazione numerica densa di un oggetto (testo, immagine, audio) in uno spazio vettoriale a N dimensioni, dove la **distanza geometrica** tra vettori riflette la **similarità semantica** tra gli oggetti corrispondenti.

Intuizione geometrica

Immagina uno spazio a 1536 dimensioni (tipico per text-embedding-3-small di OpenAI). In questo spazio:

- "cane" e "gatto" sono vicini — stessa categoria semantica.
- "re" – "uomo" + "donna" $\approx$ "regina" — operazioni aritmetiche significative.
- "Python" e "JavaScript" sono vicini; "Python" e "serpente" sono in una zona diversa (disambiguazione contestuale durante il training).

Come si generano gli embeddings

Un modello encoder (es. BERT, sentence-transformers) processa il testo e restituisce il vettore dello strato [CLS] o una media dei token. I modelli dedicati agli embeddings sono ottimizzati con:

- **Contrastive Learning:** coppie (anchor, positivo) vengono avvicinate; coppie (anchor, negativo) allontanate nello spazio vettoriale.
- **Multiple Negatives Ranking Loss:** loss efficiente su grandi batch dove ogni sample è negativo per gli altri.

Metriche di similarità

Cosine Similarity	Angolo tra vettori — invariante alla magnitudine. Standard per testo. Range [-1, 1]
Dot Product	Prodotto scalare — veloce, sensibile alla magnitudine. Usato in MIPS retrieval
Euclidean (L2)	Distanza geometrica. Meno comune per testo, utile per immagini
Manhattan (L1)	Somma differenze assolute — robusta agli outlier, meno usata

Principali modelli di embedding

text-embedding-3-small	OpenAI, 1536 dim, ottimo rapporto qualità/costo, API
text-embedding-3-large	OpenAI, 3072 dim, massima qualità OpenAI, API
text-embedding-ada-002	OpenAI legacy, 1536 dim — ancora largamente usato
nomic-embed-text	Nomic AI, open, 768 dim, on-device, licenza Apache 2.0
mxbai-embed-large	MixedBread, 1024 dim, ottimo su MTEB benchmark
bge-m3	BAAI, multilingue, supporta retrieval dense+sparse+colbert
jina-embeddings-v3	Jina AI, 1024 dim, multilingue, task-specific LoRA adapters
all-MiniLM-L6-v2	sentence-transformers, 384 dim, leggero, ottimo per edge

Dimensionalità e Tradeoff

Aumentare le dimensioni migliora la capacità espressiva ma aumenta costo di storage e latenza di retrieval. Tecniche moderne come **Matryoshka Representation Learning (MRL)** permettono di troncare il vettore a dimensioni minori mantenendo buone performance — es. text-embedding-3-small supporta troncamento a 512 o 256 dim.

Late Chunking e Late Interaction

Late Chunking (Jina): invece di chuncare prima e poi embeddare, si embeddano prima i token dell'intero documento con contesto completo, poi si fanno pool sui chunk — migliora la coerenza contestuale.

ColBERT / late interaction: invece di un singolo vettore per documento, si mantengono i vettori di ogni token. La similarità query-doc si calcola con MaxSim tra ogni token query e tutti i token doc — più preciso, ma richiede più storage.

4. Vector Databases

Un **vector database** è un sistema di storage ottimizzato per memorizzare, indicizzare e interrogare vettori ad alta dimensione in modo efficiente. Supporta la **Approximate Nearest Neighbor (ANN)** search, che trova i K vettori più simili a una query in millisecondi su milioni di vettori.

Algoritmi di indicizzazione ANN

HNSW	Hierarchical Navigable Small World — grafo gerarchico, eccellente recall/speed tradeoff. Default in molti DB.
IVF	Inverted File Index — divide lo spazio in cluster (Voronoi), cerca solo nei cluster più vicini. Scala a miliardi.
PQ	Product Quantization — comprime i vettori quantizzando sotto-spazi. Riduce memory drasticamente.
IVF+PQ	Combinazione: IVF per coarse search + PQ per vettori compressi. Usato in Faiss per billion-scale.
LSH	Locality Sensitive Hashing — approccio hash-based, buono per alta dimensionalità.
ScaNN	Google: ottimizzato per hardware moderno (SIMD/AVX), altissime query-per-second.

Panoramica Vector Databases

Chroma	Open source, in-memory o persistente, perfetto per prototipi Python. Default in LangChain.
Pinecone	Managed cloud, serverless, zero-ops. Leader per produzione senza infrastruttura.
Weaviate	Open source + cloud, schema GraphQL, supporto multimodale, hybrid search nativo.
Qdrant	Open source, Rust, alta performance, payload filtering avanzato, on-prem o cloud.
Milvus	Open source, scalabilità massima (billion vectors), architettura distribuita.
pgvector	Extension PostgreSQL — ideal per chi ha già Postgres, senza infrastruttura aggiuntiva.
FAISS	Libreria Meta, non un DB completo — embedding nel codice, massima velocità pura.
Redis	Redis Stack con modulo Search/VSS — low-latency, già in stack di molte aziende.

Hybrid Search: Dense + Sparse

Il retrieval puramente denso (embedding semantico) può fallire su query con termini tecnici precisi o codici. Il **Hybrid Search** combina:

- **Dense retrieval:** embedding semantici — cattura significato e parafrasi.
- **Sparse retrieval:** BM25 / TF-IDF — match esatto su keyword, acronimi, ID prodotto.

I due score vengono fusi con **Reciprocal Rank Fusion (RRF)** o combinazione lineare pesata. Weaviate, Qdrant e Elasticsearch supportano hybrid search nativamente.

5. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) è un'architettura che aumenta un LLM con la capacità di recuperare dinamicamente informazioni rilevanti da una knowledge base esterna, inserendole nel prompt prima della generazione. Risolve i limiti principali dei LLM standalone: knowledge cutoff, hallucination su fatti specifici, contesto privato/aziendale.

Pipeline RAG standard (Naive RAG)

1. Ingestion	Caricamento documenti → chunking → embedding → storage in vector DB
2. Query	Input utente → embedding della query con lo stesso modello usato in ingestion
3. Retrieval	ANN search nel vector DB → top-K chunk più simili alla query
4. Augmentation	Costruzione del prompt: system prompt + chunk recuperati + query utente
5. Generation	LLM genera la risposta condizionata al contesto recuperato
6. (Optional)	Post-processing: citation extraction, fact-checking, re-ranking output

Strategia di Chunking

Il chunking è critico: chunk troppo piccoli perdono contesto, troppo grandi introducono rumore nel retrieval. Principali strategie:

- **Fixed-size chunking:** N token con overlap percentuale (es. 512 token, 20% overlap). Semplice, baseline.
- **Sentence splitting:** rispetta i confini di frase — buon bilanciamento. spaCy / NLTK.
- **Recursive character splitting:** LangChain default — split su paragrafo, poi frase, poi carattere.
- **Semantic chunking:** calcola l'embedding di ogni frase, crea chunk dove il cosine similarity tra frasi consecutive scende sotto soglia — chunk semanticamente coerenti.
- **Document-aware:** rispetta la struttura del documento (heading Markdown, sezioni PDF) — migliore per documenti strutturati.

Problemi del Naive RAG

- **Retrieval impreciso:** la query dell'utente può avere forma diversa da come è scritta nei documenti (vocabulary mismatch).
- **Lost in the middle:** i LLM tendono a ignorare le informazioni al centro del prompt — reranking aiuta.
- **Chunk senza contesto:** un chunk estratto dal centro di un documento perde il contesto delle sezioni precedenti.
- **Multi-hop reasoning:** domande che richiedono connettere informazioni da più documenti non risolte da un singolo retrieval.

6. Tecniche RAG Avanzate

Query Transformation

Trasformare la query prima del retrieval per migliorarne il match con i documenti:

- **HyDE (Hypothetical Document Embedding)**: si chiede al LLM di generare un documento ipotetico che risponderebbe alla query, poi si usa il suo embedding per il retrieval — più simile ai documenti reali.
- **Multi-Query**: si generano N riformulazioni diverse della query, si fa retrieval per ognuna, si deduplicano i risultati — aumenta il recall.
- **Step-Back Prompting**: genera una query più generica/astratta che recupera contesto di background utile per la risposta finale.
- **Query Decomposition**: scompone query complesse in sotto-query più semplici, risolve ciascuna, poi sintetizza.

Reranking

Dopo il retrieval ANN (recall-oriented), un **reranker** riduce il set ai K chunk più rilevanti con un modello cross-encoder (precision-oriented). I cross-encoder confrontano query e documento insieme, producendo uno score di rilevanza molto più preciso degli embeddings bi-encoder.

- **Cohere Rerank**: API cloud, leader in qualità.
- **BGE Reranker**: open source BAAI, ottimo on-premise.
- **FlashRank**: ultra-veloce, no GPU required, ottimo per latency-critical applications.

Contextual Retrieval

Tecnica Anthropic (2024): prima di embeddare ogni chunk, si chiede a un LLM di generare 1-2 frasi di contesto che spiegano dove si trova il chunk nell'intero documento. Questo testo di contesto viene preposto al chunk prima dell'embedding — migliora il retrieval del 35-49% secondo i benchmark.

Parent-Child Chunking

Si creano **chunk piccoli** (child) per un retrieval più preciso, ma quando un child chunk viene recuperato, si inserisce nel prompt il suo **chunk parent** (più grande) per avere contesto completo. Implementato nativamente in LangChain come *ParentDocumentRetriever*.

Graph RAG

Microsoft Research (2024): invece di un semplice vector DB, si costruisce un **knowledge graph** dal corpus dove nodi = entità e archi = relazioni. Il retrieval naviga il grafo per rispondere a domande che richiedono connettere più entità. Particolarmente efficace per query globali su grandi corpus.

Agentic RAG

Un LLM agent decide **quando e come** fare retrieval in modo iterativo:

- **FLARE**: genera la risposta progressivamente; quando la confidence scende, fa retrieval attivo per completare.
- **Self-RAG**: il LLM è addestrato a inserire token speciali per decidere se fare retrieval, valutare la rilevanza dei chunk, e criticare la propria risposta.
- **Corrective RAG (CRAG)**: valuta i chunk recuperati; se irrilevanti, fa web search come fallback.

7. Valutazione e Best Practice

Metriche di Retrieval

Precision@K	Frazione dei K chunk recuperati che sono effettivamente rilevanti
Recall@K	Frazione dei chunk rilevanti totali che sono stati recuperati nei top-K
MRR	Mean Reciprocal Rank — posizione media del primo chunk rilevante
NDCG@K	Normalized Discounted Cumulative Gain — considera la posizione nel ranking
Hit Rate	% delle query per cui almeno un chunk rilevante è nei top-K

Framework di Valutazione RAG

- **RAGAS**: framework open source specifico per RAG — misura faithfulness (risposta fedele ai chunk), answer relevancy, context precision, context recall.
- **ARES**: Automated RAG Evaluation System — allena classificatori LLM-based per la valutazione.
- **TruLens**: dashboard di valutazione e monitoring per applicazioni LLM, supporta RAG triad: context relevance, groundedness, answer relevance.

Best Practice — Checklist

- **Scegli il modello di embedding appropriato**: stesso modello in ingestion e retrieval. Per italiano, considera modelli multilingue (bge-m3, jina-v3).
- **Normalizza il testo**: lower-case, rimozione caratteri speciali, gestione unicode prima dell'embedding.
- **Aggiungi metadati ai chunk**: titolo documento, sezione, data, autore — permettono filtering per metadata al momento del retrieval.
- **Valuta il chunking empiricamente**: non esiste un chunk size universale — testa 256/512/1024 token sul tuo dominio.
- **Implementa un fallback**: se il retrieval non trova nulla di rilevante (score sotto soglia), non passare chunk al LLM — meglio rispondere "non so" che hallucinare.
- **Logga tutto**: query, chunk recuperati, risposta finale — essenziale per debug e miglioramento iterativo.
- **Monitora la drift**: la knowledge base cambia nel tempo — reembedding periodico o incremental indexing.

Stack tecnologico consigliato

Stack RAG Production-Ready
Orchestrazione : LangChain o LlamaIndex per la pipeline RAG
Embedding : text-embedding-3-small (API) o bge-m3 (locale)
Vector DB : Qdrant (on-prem) o Pinecone (managed)
Reranker : BGE Reranker v2 o Cohere Rerank
LLM : Claude 3.5 Sonnet / GPT-4o (API) o LLaMA 3.1 70B (locale)
Valutazione : RAGAS + logging su Langfuse

Glossario Rapido

ANN	Approximate Nearest Neighbor — ricerca veloce dei vettori più simili, senza garantire i K esatti.
Attention	Meccanismo core del Transformer che pesa l'importanza di ogni token rispetto agli altri.
Chunk	Porzione di testo in cui viene diviso un documento per l'embedding e il retrieval.
Context Window	Numero massimo di token processabili da un LLM in una singola inference.
CoT	Chain of Thought — tecnica di prompting per stimolare il ragionamento step-by-step.
Dense Retrieval	Retrieval basato su embedding semantici (vettori densi).
Embedding	Rappresentazione vettoriale densa di un oggetto (testo, immagine) in spazio semantico.
Fine-tuning	Addestramento aggiuntivo di un modello pre-addestrato su dati specifici del dominio.
Foundation Model	Modello pre-addestrato su larga scala, adattabile a molti task specifici.
Hallucination	Generazione di informazioni false ma plausibili da parte di un LLM.
HNSW	Hierarchical Navigable Small World — algoritmo grafo per ANN search efficiente.
HyDE	Hypothetical Document Embedding — genera documento ipotetico per migliorare il retrieval.
Inference	Processo di utilizzo di un modello addestrato per generare output su nuovi input.
Knowledge Cutoff	Data entro cui il LLM ha visto dati di training — non conosce eventi successivi.
LLM	Large Language Model — rete neurale Transformer enormi pre-addestrata su testo.
Logit	Output grezzo (pre-softmax) del layer finale di un LLM, proporzionale alla log-probabilità.
MRL	Matryoshka Representation Learning — embeddings troncabili a dimensioni minori.
Perplexity	Metрика di valutazione LLM: misura quanto il modello è 'sorpreso' dal testo di test.
Prompt Engineering	Arte di formulare prompt per ottimizzare le risposte di un LLM.
RAG	Retrieval-Augmented Generation — integra retrieval esterno nel processo generativo del LLM.
Reranker	Modello cross-encoder che riordina i chunk per rilevanza dopo il retrieval iniziale.
RLHF	Reinforcement Learning from Human Feedback — allineamento LLM su preferenze umane.
Semantic Search	Ricerca per significato (via embedding) invece di match esatto di keyword.
Sparse Retrieval	Retrieval basato su BM25/TF-IDF — match su keyword esatte.

System Prompt	Istruzione iniziale nascosta all'utente che configura il comportamento del LLM.
Temperature	Parametro di sampling: basso = deterministico, alto = creativo/casuale.
Token	Unità base di testo per i LLM — circa 0.75 parole in inglese, ~0.6 in italiano.
Transformer	Architettura neurale basata su self-attention — base di tutti i LLM moderni.
Vector DB	Database specializzato per storage e retrieval efficiente di vettori ad alta dimensione.